



 @CafecomDadoseGatos

Sumário

OpenClaw FULL no Docker + Ollama no Windows+Whatsapp+Telegram — Passo a passo completo (com fix do 1008)	2
1) Pré-requisitos no Windows	2
2) Instalar OpenClaw FULL no WSL2 (Ubuntu)	2
3) Subir e acessar a UI	3
4) Validar conectividade do container com o Ollama do Windows	3
5) Problema: 1008 “device token mismatch” / “pairing required”	4
6) Aplicar patch no ~/.openclaw/openclaw.json (anti-1008 + Ollama)	4
7) Reiniciar gateway (compose FULL)	6
8) Health check oficial.....	6
9) Testar se OpenClaw fala com Ollama (endpoint OpenAI /v1)	7
10) Teste final na UI	7
11) WhatsApp + Telegram (via onboard)	8
12) Rodar onboard (FULL).....	8

OpenClaw FULL no Docker + Ollama no Windows+Whatsapp+Telegram — Passo a passo completo (com fix do 1008)

Objetivo

Rodar o **OpenClaw (full)** dentro do **Docker** (via WSL2/Ubuntu) e usar o **Ollama no Windows** como provedor de modelo — **tudo local, com token, sem cloud e custo zero.**

1) Pré-requisitos no Windows

1.1 Instalar e iniciar o Ollama

Instale o Ollama no Windows e confirme que o serviço está rodando.

1.2 Baixar o modelo (exemplo)

```
ollama pull qwen3:8b
```

Tem short no canal mostrando como instalar o Ollama

1.4 Confirmar API do Ollama

Use curl.exe (pra evitar prompt do PowerShell):

```
curl.exe http://127.0.0.1:11434/api/tags
```

2) Instalar OpenClaw FULL no WSL2 (Ubuntu)

No Ubuntu/WSL2:

2.1 Clonar repo e rodar o setup FULL

```
cd ~  
rm -rf ~/openclaw  
git clone https://github.com/openclaw/openclaw.git  
cd openclaw  
chmod +x ./docker-setup.sh  
./docker-setup.sh
```

2.2 Onboarding

Quando perguntar no wizard, escolher:

- **Gateway bind:** lan
- **Gateway auth:** token
- **Tailscale exposure:** Off
- **Install Gateway daemon:** No
- **Model/auth provider:** Skip for now (vamos configurar Ollama depois)

Ao final, ele cria/atualiza:

- Config: ~/.openclaw/openclaw.json
- Compose FULL: /home/denise/openclaw/docker-compose.yml
- Container principal: openclaw-gateway (via compose)

3) Subir e acessar a UI

O gateway sobe mapeado na máquina (host):

- UI: http://localhost:18789/
- UI com token: http://localhost:18789/#token=SEU_TOKEN

Sempre use a URL **com token**.

4) Validar conectividade do container com o Ollama do Windows

No WSL:

```
cd ~/openclaw
docker compose exec openclaw-gateway sh -lc 'curl -sS
http://host.docker.internal:11434/api/version && echo'
```

Saída esperada:

```
{"version": "..."} 
```

✅ Isso prova que o container alcança o Ollama no Windows.

5) Problema: 1008 “device token mismatch” / “pairing required”

Sintomas na UI:

- “Disconnected from gateway”
- disconnected (1008): unauthorized: device token mismatch
- disconnected (1008): pairing required

Causa típica em Docker Desktop + WSL2:

- A UI chega no gateway via IP interno (NAT / proxy do Docker) e o gateway exige pairing / não confia no proxy.

✅ Solução estável:

- token auth (já temos)
- habilitar control UI no config
- permitir auth “insecure” **apenas para UI local**
- adicionar subnets do Docker em gateway.trustedProxies
- configurar provider do Ollama no openclaw.json + modelo padrão

6) Aplicar patch no ~/.openclaw/openclaw.json (anti-1008 + Ollama)

6.1 Instalar jq

```
sudo apt-get update && sudo apt-get install -y jq
```

6.2 Patch completo (com backup + validação)

```
set -euo pipefail

# backup
cp -a ~/.openclaw/openclaw.json
~/.openclaw/openclaw.json.bak.$(date +%Y%m%d-%H%M%S)

TOKEN="$(jq -r '.gateway.auth.token'
~/.openclaw/openclaw.json)"
echo "TOKEN(JSON)={$TOKEN:0:8}...${TOKEN: -6}"
```

```
jq --arg TOKEN "$TOKEN" '
  .gateway.auth.mode="token"
  | .gateway.auth.token=$TOKEN
  | .gateway.controlUi.enabled=true
  | .gateway.controlUi.allowInsecureAuth=true
  | .gateway.trustedProxies = ((.gateway.trustedProxies //
[])
    +
["192.168.65.0/24","172.17.0.0/16","172.18.0.0/16","172.19.
0.0/16"]
    | unique)
  | .models.providers.ollama = {
    "baseUrl":"http://host.docker.internal:11434/v1",
    "apiKey":"ollama-local",
    "api":"openai-responses",

"models": [{ "id": "qwen3:8b", "name": "qwen3:8b", "contextWindow
": 32768 }]
  }
  | .agents.defaults.model.primary = "ollama/qwen3:8b"
' ~/.openclaw/openclaw.json > ~/.openclaw/openclaw.json.tmp

jq -e . ~/.openclaw/openclaw.json.tmp >/dev/null
mv ~/.openclaw/openclaw.json.tmp ~/.openclaw/openclaw.json
echo "✅ patched ~/.openclaw/openclaw.json"
```

6.3 Conferir config

```
jq -r '.gateway.auth.mode, .gateway.auth.token'
~/.openclaw/openclaw.json
jq -c '.gateway.trustedProxies' ~/.openclaw/openclaw.json
jq -r '.models.providers.ollama.baseUrl,
.agents.defaults.model.primary' ~/.openclaw/openclaw.json
```

Esperado:

- token
- token completo
- trustedProxies com 172.17/16, 172.18/16, 172.19/16, 192.168.65/24
- baseUrl http://host.docker.internal:11434/v1

- primary ollama/qwen3:8b

7) Reiniciar gateway (compose FULL)

```
cd ~/openclaw
```

```
docker compose restart openclaw-gateway  
docker compose logs -f --tail 120 openclaw-gateway
```

Procure nos logs:

- agent model: ollama/qwen3:8b
- webchat connected ... client=openclaw-control-ui

✅ Isso confirma que o 1008 foi resolvido.

8) Health check oficial

Nesta versão, o comando `node dist/index.js health --token ...` **não aceita** `--token` (dá unknown option '--token').

Então usamos health checks que funcionam sempre.

8.1 Health HTTP (rápido)

```
curl -fsS http://localhost:18789/__openclaw__/canvas/  
>/dev/null && echo "HTTP OK"
```

8.2 Health WS + Token (prova de auth)

Instalar wscat via apt:

```
sudo apt-get update  
sudo apt-get install -y node-ws
```

Conectar:

```
TOKEN="$(jq -r '.gateway.auth.token'  
~/openclaw/openclaw.json) "  
wscat -c ws://localhost:18789 -H "Authorization: Bearer  
$TOKEN"
```

Saída esperada:

Connected (press CTRL+C to quit)

```
<{"type":"event","event":"connect.challenge","payload":{"nonce":"...","ts":...}}
>
```

9) Testar se OpenClaw fala com Ollama (endpoint OpenAI /v1)

Teste direto dentro do container:

```
cd ~/openclaw

docker compose exec openclaw-gateway sh -lc '
curl -sS
http://host.docker.internal:11434/v1/chat/completions \
  -H "Content-Type: application/json" \
  -d
"{\"model\":\"qwen3:8b\", \"messages\": [{\"role\":\"user\", \"content\":\"Responda com 1 palavra: OK\"}], \"temperature\":0, \"stream\":false}" \
| head -c 400 && echo
'
```

Saída esperada (trecho):

```
{"model":"qwen3:8b", ... "content":"OK" ...}
```

✓ Isso prova que:

- o container acessa o Ollama
 - o endpoint OpenAI compatível está OK
 - o modelo responde
-

10) Teste final na UI

Abra:

http://localhost:18789/#token=SEU_TOKEN

Pergunte:

“Diga Oi”.

11) WhatsApp + Telegram (via onboard)

Esse trecho é pra “habilitar canais” no FULL.

Backup antes de mexer (recomendado)

```
set -euo pipefail
TS="$(date +%Y%m%d-%H%M%S) "
mkdir -p ~/.openclaw/backup-$TS

cp -a ~/.openclaw/openclaw.json ~/.openclaw/backup-$TS/
cp -a /home/denise/openclaw/docker-compose.yml
~/.openclaw/backup-$TS/ 2>/dev/null || true
cp -a ~/.openclaw/workspace ~/.openclaw/backup-$TS/
2>/dev/null || true

echo "☑ backup em ~/.openclaw/backup-$TS"
```

12) Rodar onboard (FULL)

```
cd ~/openclaw
docker compose run --rm openclaw-cli onboard
```

No wizard:

- **Configure chat channels now?** → Yes
- Selecciona **Telegram (Bot API)** → cola SEU_TOKEN_TELEGRAM
- Selecciona **WhatsApp (QR link)** → Yes → escaneia QR
- DM policy Telegram → Pairing (recommended)
- Skills → No
- Hooks → Skip for now

O openclaw.json FINAL dentro do container (com WhatsApp + Telegram)

(Esse é o comando “só ler”, sem instalar nada dentro do container.)


```
cd ~/openclaw
docker compose exec openclaw-gateway sh -lc '
sed -n "1,260p" /home/node/.openclaw/openclaw.json
'
```

Você deve ver:

- channels.whatsapp ...
- channels.telegram ...
- plugins.entries.telegram.enabled=true
- plugins.entries.whatsapp.enabled=true
- gateway.auth.token ...
- trustedProxies ...
- models.providers.ollama.baseUrl ...
- agents.defaults.model.primary = ollama/qwen3:8b